

Condicionais

Tomar diferentes ações a depender de certos fatores está no cerne da programação e da robótica. "Caso haja obstáculo, pare, se não, vá para frente" é uma estrutura lógica comum em aplicações de robôs móveis.

Nesta aula estudaremos condicionais e diferentes formas de tomarmos decisões em código. As ferramentas a serem usadas são: `If`, `Else`, `Else if`, `Switch` e `Operadores ternários`.

```
In [ ]: # Rode o comando abaixo para poder usar os códigos em cpp

!mkdir cpp_files
```

Para rodar os exemplos desta aula, abra um terminal do Vscodé, navegue até a pasta da aula e rode o comando `./cpp_files/(nome gerado do exemplo)`

If - "Se"

Usa-se a declaração `If` para especificar um bloco de comandos a serem executados caso uma condição seja verdadeira. A declaração deve ser feita na seguinte forma:

```
if(CONDICIONAL){
    (Código a ser executado caso verdadeiro)
}
```

Abaixo podemos ver um exemplo básico de sua aplicação:

```
In [19]: %%writefile cpp_files/primeiro_exemplo.cpp
#include <iostream>
using namespace std;

int main(){
    cout<<"----- Checador de maioridade -----"<<endl;
    cout<<"Entre a sua idade: ";
    int idade;
    cin>>idade;
    if(idade>=18){
        cout<<"Você é maior de idade"<<endl;
    }
    if(idade<18){
        cout<<"Você é menor de idade"<<endl;
    }
    return 0;
}
```

Overwriting cpp_files/primeiro_exemplo.cpp

```
In [1]: !g++ cpp_files/primeiro_exemplo.cpp -o cpp_files/primeiro_exemplo
```

Para rodar o exemplo, abra um terminal no Vscode, navegue até a pasta da aula e rode o comando `./cpp_files/primeiro_exemplo`

Else - "Se não"

Tendo entendido a declaração de "Se", a declaração de "Se não", `Else` deve vir de forma natural. Ela deve vir acompanhada de um bloco `If`, já que não faz sentido falar "Se não" sem um "Se", e dentro de seu corpo deve-se por o código a ser executado caso a condição do `If` seja falsa.

Vamos analisar o exemplo anterior, com o uso de `Else`:

```
In [1]: %%writefile cpp_files/segundo_exemplo.cpp
#include <iostream>
using namespace std;

int main(){
    cout<<"----- Checador de maioridade -----"<<endl;
    cout<<"Entre a sua idade: ";
    int idade;
    cin>>idade;
    if(idade>=18){
        cout<<"Você é maior de idade"<<endl;
    }else{
        cout<<"Você é menor de idade"<<endl;
    }
    return 0;
}
```

Overwriting `cpp_files/segundo_exemplo.cpp`

```
In [2]: !g++ cpp_files/segundo_exemplo.cpp -o cpp_files/segundo_exemplo
```

Else If - "Se não, se"

Esta declaração é utilizada quando queremos especificar novas condições caso a primeira seja falsa. O `Else if` deve ser acompanhado de um bloco `If` e, geralmente, vai possuir também um bloco `Else` ao final.

A declaração deve ser feita de forma semelhante ao `If`, com uma condicional e o código a ser executado caso a condição seja verdadeira.

Vamos analisar um código usando `If`, `Else If` e `Else`

```
In [23]: %%writefile cpp_files/terceiro_exemplo.cpp
#include <iostream>
```

```
using namespace std;

int main(){
    cout<<"----- Checador de maioridade/Idoso -----"<<endl;
    cout<<"Entre a sua idade: ";
    int idade;
    cin>>idade;
    if(idade<18){
        cout<<"Você é menor de idade"<<endl;
    }else if(idade<60){
        cout<<"Você é maior de idade e não é idoso"<<endl;
    }else{
        cout<<"Você é maior de idade e é idoso"<<endl;
    }
    return 0;
}
```

Writing cpp_files/terceiro_exemplo.cpp

In [24]: `!g++ cpp_files/terceiro_exemplo.cpp -o cpp_files/terceiro_exemplo`

Note que não há limite para a quantidade de `Else ifs` que você pode por em uma única estrutura lógica, mas é fato que pode ser que o código fique confuso, caso haja muita demanda por checagens.

Por isso, para checagens com muitas possibilidades, usa-se a estrutura do `Switch`, que veremos a seguir.

Switch - Várias checagens juntas

Como dito acima, usamos a declaração `Switch` quando queremos ter várias checagens. A estrutura segue a seguinte lógica: Deve-se entrar uma variável a ser analisada, haverão checagens do valor na variável e serão executados códigos baseado nesse valor. Confuso? Vamos analisar um exemplo para esclarecer mais essa declaração:

In [25]: `%writefile cpp_files/quarto_exemplo.cpp`

```
#include <iostream>
using namespace std;

int main(){
    cout<<"----- Calculadora simples -----"<<endl;
    cout<<"Entre o primeiro numero: ";
    float num1;
    cin>>num1;
    cout<<"Entre o segundo: ";
    float num2;
    cin>>num2;
    cout<<"Entre a operação desejada (+,-,*,/): ";
    char op;
    cin>>op;
    switch(op){
        case '+':
```

```

        cout<<"Resultado: "<<num1+num2<<endl;
        break;
    case '-':
        cout<<"Resultado: "<<num1-num2<<endl;
        break;
    case '*':
        cout<<"Resultado: "<<num1*num2<<endl;
        break;
    case '/':
        cout<<"Resultado: "<<num1/num2<<endl;
        break;
    default:
        cout<<"Operação inválida"<<endl;
    }
    return 0;
}

```

Writing cpp_files/quarto_exemplo.cpp

In [26]: `!g++ cpp_files/quarto_exemplo.cpp -o cpp_files/quarto_exemplo`

Viu como não é um bicho de 7 cabeças? A estrutura do `Switch` é como no exemplo, com os casos diferentes e a utilização de um `default`, caso o valor da variável não se enquadre em nenhum caso. Um ponto importante para a utilização dessa declaração é o operador `break`, que deve estar presente em cada um dos casos, uma vez que ele garante que o código não irá para outro bloco dentro do `Switch`. (Altere o código retirando o `break` para ver o resultado!)

Há, porém, vezes com o uso do `Switch`.

- Deve-se usar apenas `Int`, `Char` ou uma variável criada com `Enum`.
- Não podem haver casos duplicados.
- O uso do `break` é opcional, mas o código executará casos errados até encontrar um.

Veja abaixo uma tabela comparativa sobre o uso de `If - Else If` e `Switch`.

switch	if else if
Executa diferentes casos com base no valor da variável do switch.	Executa diferentes blocos com base na condição especificada.
Só pode avaliar expressões do tipo <code>Int</code> , <code>Char</code> ou <code>Enum</code> .	Pode avaliar qualquer tipo de expressão.
Mais rápido e fácil de ler quando há um grande número de condições.	Pode ficar confuso quando há muitas condições.

Ifs encadeados

É possível declarar um `If` dentro de outro, gerando o que chamamos de "Ifs encadeados". Nessa estrutura, a segunda checagem só será chamada caso a primeira seja verdadeira. Abaixo, podemos ver um exemplo de sua aplicação:

```
In [7]: %%writefile cpp_files/quinto_exemplo.cpp
#include <iostream>
using namespace std;

int main(){
    cout<<"----- Checador de maioridade/Idoso/Bebê -----"<<endl;
    cout<<"Entre a sua idade: ";
    int idade;
    cin>>idade;
    if(idade>=18){
        if(idade<60){
            cout<<"Você é maior de idade e não é idoso"<<endl;
        }else{
            cout<<"Você é maior de idade e é idoso"<<endl;
        }
    }else{
        if(idade>=2){
            cout<<"Você é menor de idade e criança"<<endl;
        }else{
            cout<<"Você é menor de idade e bebê"<<endl;
        }
    }
    return 0;
}
```

Overwriting cpp_files/quinto_exemplo.cpp

```
In [8]: !g++ cpp_files/quinto_exemplo.cpp -o cpp_files/quinto_exemplo
```

Note que você pode, novamente, utilizar quantos `Ifs` encadeadores quanto necessários, mas evite ter muitas "camadas", pois elas dificultam o entendimento do código de forma geral.

Operadores lógicos em condições

É possível combinar ou reverter condições utilizando operadores lógicos, permitindo maior complexidade em blocos de `If`, `Else` e `Else if`. Os operadores a serem utilizados são:

- `&&` (AND) - Todas as condições devem ser verdadeiras
- `||` (OR) - Pelo menos uma condição deve ser verdadeira
- `!` (NOT) - Inverte a condição (true -> false, false -> true)

```
In [5]: %%writefile cpp_files/sexta_exemplo.cpp
#include <iostream>
using namespace std;

int main(){
    bool condicao_1, condicao_2;
    condicao_1 = false;
    condicao_2 = true;
```

```

if(condicao_1 && condicao_2){
    cout<<"As duas são verdadeiras"<<endl;
}
if(condicao_1 || condicao_2){
    cout<<"Pelo menos uma é verdadeira"<<endl;
}
cout<<"A condição 1 é "<<condicao_1<<" , a negação dela é "<<!condicao_1<<endl;
cout<<"A condição 2 é "<<condicao_2<<" , a negação dela é "<<!condicao_2<<endl;
}

```

Overwriting cpp_files/sexta_exemplo.cpp

In [6]: `!g++ cpp_files/sexta_exemplo.cpp -o cpp_files/sexta_exemplo`

Yoda Conditions: Ao contrario falarei para problemas evitar

Atribuições e comparações estão, muitas vezes, a um caracter de distância. É um erro totalmente plausível escrever `var = 67` ao invés de `var == 67` ao escrever uma condição de um `If`. Mas, como o bater de asas de uma borboleta pode desencadear um furacão, esse erro pode trazer problemas gigantescos em seu código. A operação, que antes era de comparação, virou de atribuição, o que significa que:

- A variável `var` recebe o valor de 67, o que não é esperado.
- O bloco do `If` será acionado (caso a atribuição errada não seja 0.)

Rode o código abaixo e altere os valores para testar as consequências.

```

In [ ]: %%writefile cpp_files/setimo_exemplo.cpp
#include <iostream>
using namespace std;

int main(){
    int var = 10;
    if(var == 10){
        cout<<"É 10"<<endl;
    }
    if(var = 67){
        cout<<"67..."<<endl;
    }
    cout<<"var agora é: "<<var<<endl;
}

```

Overwriting cpp_files/setimo_exemplo.cpp

In [22]: `!g++ cpp_files/setimo_exemplo.cpp -o cpp_files/setimo_exemplo`

Com isso em mente, alguns programadores desenvolveram as chamadas **Yoda Conditions**, inspiradas no modo de falar do mestre Yoda, de Star Wars. A técnica consiste em inverter as posições da variável e do valor de checagem em um bloco de condição, escrevendo algo como `67 == var`. Essa técnica é permissiva com o erro com o

qual estamos tratando, uma vez que o código retornará um erro caso você esqueça um dos símbolos de `=`, já que não é possível realizar uma atribuição em uma "não variável".

Abaixo temos o exemplo anterior, mas escrito utilizando `Yoda Conditions`, note a diferença de comportamento entre os dois.

```
In [ ]: %%writefile cpp_files/oitavo_exemplo.cpp
#include <iostream>
using namespace std;

int main(){
    int var = 10;
    if(10 == var){
        cout<<"É 10"<<endl;
    }
    if(67 = var){
        cout<<"67..."<<endl;
    }
    cout<<"var agora é: "<<var<<endl;
}
```

Overwriting cpp_files/oitavo_exemplo.cpp

```
In [26]: !g++ cpp_files/oitavo_exemplo.cpp -o cpp_files/oitavo_exemplo
```

Guard Clauses

Quando começamos a escrever condicionais mais complexas, é comum cair em vários ifs encadeados, um dentro do outro. O código funciona, mas pode ficar cada vez mais difícil de ler e manter. Em situações assim, muitos programadores usam a técnica de Guard Clauses.

Guard Clauses são verificações feitas logo no início de uma função, interrompendo sua execução quando algum requisito não é atendido. Nesta aula ainda não estudaremos funções em profundidade, mas já vale a pena conhecer essa ideia, porque ela aparece bastante em códigos reais.

A lógica é simples: ao invés de colocar toda a parte principal do código dentro de vários blocos aninhados, fazemos checagens curtas no começo e usamos return para sair da função assim que detectamos um problema. Isso costuma deixar o código mais fácil de ler, além de facilitar a manutenção, principalmente em programas mais longos e com muitas decisões.

Vejamos um exemplo:

```
#include <iostream>
using namespace std;

void liberarAcesso(bool login, bool senha){
```

```

    if(login){
        if(senha){
            cout<<"Acesso permitido"<<endl;
        }else{
            cout<<"Senha invalida"<<endl;
        }
    }else{
        cout<<"Login invalido"<<endl;
    }
}

int main(){
    bool login = true;
    bool senha = false;

    liberarAcesso(login, senha);

    return 0;
}

```

Note como, nesse caso, os 2 ifs encadeados, juntamente com seus "elses", geram um código relativamente complexo de se ler. Imagine então quando adicionamos mais camadas de verificação, pode ficar ainda mais confuso e difícil de manter. É aí que o conceito de Guard Clauses brilha, vamos ver o exemplo abaixo:

```

#include <iostream>
using namespace std;

void liberarAcesso(bool login, bool senha){
    if(!login){
        cout<<"Login invalido"<<endl;
        return;
    }

    if(!senha){
        cout<<"Senha invalida"<<endl;
        return;
    }

    cout<<"Acesso permitido"<<endl;
}

int main(){
    bool login = true;
    bool senha = false;

    liberarAcesso(login, senha);

    return 0;
}

```

Nesse exemplo, a função primeiro verifica se o login é válido. Caso não seja, ela já imprime a mensagem e encerra. Depois, verifica a senha. Apenas se todas as condições forem

atendidas o acesso é liberado.

Ambos os códigos funcionam, mas a versão com Guard Clauses tende a ficar mais organizada quando o número de verificações cresce. Em exemplos pequenos, a diferença pode parecer pouca. Porém, em funções maiores, com muitas regras, permissões e validações, evitar vários níveis de aninhamento deixa o código muito mais limpo.

Operador ternário - If - Else compacto

Operadores ternários são declarações que podem ser utilizadas para substituir diversas linhas, uma vez que realizam checagens equivalentes a um `If` e um `Else`, retornando um resultado baseado em uma condição.

A sintaxe de operadores ternários é:

```
var = (condição) ? (Valor caso real) : (Valor caso falso);
```

Operadores ternários são muito utilizados em atribuições de variáveis e impressões na tela, como nos exemplos abaixo.

Vale lembrar que é possível ter operadores ternários encadeados, mas isso diminui muito a capacidade de leitura do código, sendo indicado utilizar estruturas mais comuns de `If - Else`.

```
In [1]: %%writefile cpp_files/nono_exemplo.cpp
#include <iostream>
using namespace std;

int main(){
    int nota;
    cout<<"Insira sua nota: ";
    cin>>nota;
    string passou_ou_nao = ((nota>=5) ? "Passou" : "Não passou");
    cout<<passou_ou_nao<<endl;

    //Poderia ser também cout<<((nota>=5) ? "Passou" : "Não passou")<<endl;
}
```

Overwriting cpp_files/nono_exemplo.cpp

```
In [2]: !g++ cpp_files/nono_exemplo.cpp -o cpp_files/nono_exemplo
```

Resumo e finalização

Nesta aula aprendemos como controlar o fluxo de execução de um programa usando estruturas condicionais. Vimos como o C++ permite tomar decisões com:

- `if` – executar algo somente se uma condição for verdadeira

- else – caminho alternativo caso a condição seja falsa
- else if – testar múltiplas condições em sequência
- switch – alternativa organizada para muitos casos discretos
- ifs encadeados – colocar condições dentro de outras
- operadores lógicos (&&, ||, !) – combinar condições
- operador ternário – forma compacta de escrever if-else simples
- yoda conditions – estilo de código diferente

Essas ferramentas formam a base de praticamente toda lógica de programação, e serão essenciais quando trabalharmos com sensores, atuadores e sistemas robóticos usando C++ + ROS.

Faça a lista de exercícios desta aula e não hesite em tirar dúvidas!